Operating Systems: Complete Notes

Unit 1: Introduction to Operating Systems

1.1 What is an Operating System?

An OS is system software that manages hardware and provides services to application programs. It acts as an **intermediary between the user and the hardware**.

Goals: convenience (easy to use), efficiency (good resource use), ability to evolve.

1.2 Main functions

- **Process management:** create, schedule, terminate processes.
- **Memory management:** allocate and free memory, virtual memory.
- **File system management:** files, directories, permissions.
- **I/O device management:** drivers, buffering, spooling.
- **Security and protection:** access control, authentication.
- **Resource allocation and accounting.**

1.3 Types of operating systems

Type	Description	Example
Batch	Jobs grouped and run without user interaction	Early IBM systems
Multiprogramming	Several programs in memory; CPU switches when one waits for I/O	
Time-sharing (multitasking)	CPU time sliced quickly among users	Unix, Linux
Real-time (RTOS)	Guaranteed response within deadlines. **Hard** (missing a deadline is failure) and **soft**	VxWorks, FreeRTOS
Distributed	Many computers appear as one system	
Network	Provides network services to clients	
Embedded	Dedicated devices	Android Things
Mobile	Phones and tablets	Android, iOS

1.4 Kernel and structures

The **kernel** is the core part of the OS, running in privileged mode.

- **Monolithic kernel:** all services in kernel space (Linux). Fast, but a bug can crash everything.
- **Microkernel:** only minimal services (IPC, scheduling, basic memory) in the kernel; others run in user space (MINIX, QNX). More reliable, slower due to message passing.
- **Hybrid kernel:** mix of both (Windows NT, macOS).
- **Layered, modular** approaches also exist.

1.5 User mode and kernel mode

The CPU has a **mode bit**. User programs run in **user mode** (restricted). The OS runs in **kernel mode** (all instructions allowed). Privileged instructions (I/O, changing memory maps) can only run in kernel mode. Switching occurs via **system calls**, **interrupts** and **traps**.

1.6 System calls

The interface between a process and the OS. Categories:

- Process control (`fork`, `exec`, `exit`, `wait`)
- File management (`open`, `read`, `write`, `close`)
- Device management (`ioctl`)
- Information maintenance (`getpid`, `time`)
- Communication (`pipe`, `shmget`, `socket`)

Interrupt: asynchronous signal from hardware. **Trap/exception:** synchronous, caused by a program (divide by zero, system call).

1.7 Boot process

Power on, BIOS/UEFI runs POST, loads the **bootloader** (GRUB), which loads the **kernel**, which initializes devices and starts the first process (`init`/`systemd`).

Unit 2: Processes and Threads

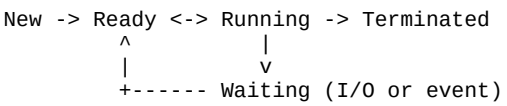
2.1 Process

A **program** is a passive file on disk. A **process** is a program **in execution**.

Memory layout of a process: Text (code), Data (globals), Heap (dynamic memory, grows up), Stack (function calls and locals, grows down).

2.2 Process states

New, Ready, Running, Waiting (Blocked), Terminated.



- Ready to Running: **dispatch** (scheduler picks it).
- Running to Ready: **preemption** (time slice over or higher priority arrives).
- Running to Waiting: waits for I/O.
- Waiting to Ready: I/O completes.

2.3 Process Control Block (PCB)

Data structure holding everything about a process: PID, state, program counter, CPU registers, scheduling info, memory-management info, accounting info, open files and I/O status.

2.4 Context switch

Saving the state of the running process in its PCB and loading the saved state of the next one. It is **pure overhead**: no useful work is done while switching.

2.5 Schedulers

- **Long-term (job) scheduler:** decides which jobs enter memory; controls degree of multiprogramming.
- **Short-term (CPU) scheduler:** picks the next ready process; runs very frequently.
- **Medium-term scheduler: swaps** processes in and out of memory.

2.6 Process creation and termination (Unix)

```
pid_t pid = fork();       // creates a child (copy of parent)
if (pid == 0) {
    execlp("ls", "ls", NULL);   // child replaces its image
} else {
    wait(NULL);                 // parent waits for the child
}
```

- `fork()` returns 0 in the child and the child's PID in the parent.
- **Zombie process:** terminated, but the parent has not yet called `wait()`; its PCB entry remains.
- **Orphan process:** parent terminated first; adopted by `init`.

Counting processes: n successive `fork()` calls (with no conditions) create 2^n - 1 child processes (2^n total).

2.7 Inter-Process Communication (IPC)

- **Shared memory:** fast; processes need to synchronize.
- **Message passing:** `send` / `receive`; easier, slower. Can be direct/indirect, blocking/non-blocking.
- **Pipes** (ordinary and named/FIFO), **message queues**, **sockets**, **signals**.

2.8 Threads

A **thread** is a lightweight unit of execution within a process. Threads of one process **share** code, data, heap and open files, but each has its own **stack, registers and program counter**.

Process	Thread
Heavyweight	Lightweight
Separate address space	Shared address space
Costly context switch	Cheap context switch
Isolated	Can affect siblings

Benefits: responsiveness, resource sharing, economy, use of multiple cores.

Models: many-to-one, one-to-one (Linux, Windows), many-to-many.

User-level threads are managed by a library (fast; one blocking call can block all). **Kernel-level threads** are managed by the OS (true parallelism).

Unit 3: CPU Scheduling

3.1 Terminology

- **Arrival time (AT):** when the process enters the ready queue.
- **Burst time (BT):** CPU time needed.
- **Completion time (CT):** when it finishes.
- **Turnaround time (TAT)** = CT - AT
- **Waiting time (WT)** = TAT - BT
- **Response time** = first time on CPU - AT
- **Throughput:** processes completed per unit time.
- **CPU utilization:** percentage of time the CPU is busy.

Goals: maximize CPU utilization and throughput; minimize TAT, WT and response time.

Preemptive: OS can take the CPU away. **Non-preemptive:** process keeps CPU until it finishes or blocks.

3.2 Algorithms

FCFS (First Come First Served): non-preemptive; simple queue. Suffers from the **convoy effect** (short jobs stuck behind a long one).

SJF (Shortest Job First): non-preemptive; picks the shortest burst. **Optimal** for minimizing average waiting time. Problem: needs burst prediction, and long jobs can **starve**.

SRTF (Shortest Remaining Time First): preemptive version of SJF.

Priority scheduling: the highest priority runs first (preemptive or not). Problem: **starvation**; solved by **aging** (gradually raise priority of waiting processes).

Round Robin (RR): each process gets a **time quantum**; then goes to the back of the queue. Preemptive, fair, good response time.

- Very large quantum behaves like FCFS; very small quantum means too many context switches.

Multilevel Queue: separate queues (system, interactive, batch) with fixed priority between queues.

Multilevel Feedback Queue (MLFQ): processes can move between queues based on behavior; the most flexible.

3.3 Worked example

Process	AT	BT
P1	0	5
P2	1	3
P3	2	8
P4	3	6

FCFS Gantt: P1 (0-5), P2 (5-8), P3 (8-16), P4 (16-22)

P	CT	TAT = CT - AT	WT = TAT - BT
P1	5	5	0
P2	8	7	4
P3	16	14	6
P4	22	19	13

Average WT = (0 + 4 + 6 + 13) / 4 = **5.75**.

SJF (non-preemptive): P1 (0-5), then among P2, P3, P4 pick shortest: P2 (5-8), P4 (8-14), P3 (14-22).

Waiting times: P1 = 0, P2 = 5 - 1 = 4, P4 = 8 - 3 = 5, P3 = 14 - 2 = 12. Average WT = (0 + 4 + 12 + 5) / 4 = **5.25**, which is better than FCFS (5.75).

3.4 Comparison

Algorithm	Preemptive	Starvation	Notes
FCFS	No	No	Convoy effect
SJF	No	Yes	Optimal avg WT
SRTF	Yes	Yes	
Priority	Both	Yes	Use aging
RR	Yes	No	Needs good quantum
MLFQ	Yes	Possible	Flexible

Unit 4: Process Synchronization

4.1 Race condition and critical section

A **race condition** occurs when the result depends on the order in which processes access shared data. The **critical section** is the code that accesses shared resources.

Example: two threads run `counter++` (load, add, store). Interleaving can lose an update.

4.2 Requirements for a solution

1. **Mutual exclusion:** only one process in its critical section.
2. **Progress:** if no one is inside, the decision of who enters cannot be postponed indefinitely.
3. **Bounded waiting:** a limit on how long a process waits to enter.

4.3 Peterson's solution (two processes)

```
// shared: bool flag[2]; int turn;
// process i (j = 1 - i)
flag[i] = true;
turn = j;
while (flag[j] && turn == j);   // busy wait
   /* critical section */
flag[i] = false;
```

Satisfies all three requirements (on simple hardware without reordering).

4.4 Hardware support

Atomic instructions: **Test-and-Set**, **Compare-and-Swap**. Used to build locks (spinlocks).

4.5 Semaphores

An integer variable accessed only through two atomic operations:

- `wait(S)` (P): `while (S <= 0); S--;`
- `signal(S)` (V): `S++;`

Binary semaphore (mutex): value 0 or 1. **Counting semaphore:** counts available resources.

Mutex vs semaphore: a mutex has an owner (only the locker unlocks); a semaphore is a signaling mechanism.

Blocking implementation avoids busy waiting: a process that cannot proceed is placed in the semaphore's waiting queue.

4.6 Classic problems

Producer-Consumer (bounded buffer)

```
semaphore mutex = 1, empty = N, full = 0;

producer:                       consumer:
  wait(empty);                    wait(full);
  wait(mutex);                    wait(mutex);
  add item to buffer;             remove item from buffer;
  signal(mutex);                  signal(mutex);
  signal(full);                   signal(empty);
```

Readers-Writers: many readers may read together; a writer needs exclusive access. Variants favor readers or writers (risk of starvation).

Dining Philosophers: 5 philosophers, 5 forks; each needs two forks to eat. A naive solution can **deadlock** (everyone picks the left fork). Fixes: allow at most 4 to sit, pick both forks atomically, or use asymmetric ordering (odd picks left first, even picks right first).

4.7 Monitors

A high-level construct: shared data plus procedures, where **only one process is active inside at a time**. Uses **condition variables** (`wait`, `signal`). Java's `synchronized`, `wait()` and `notify()` are monitor-based.

4.8 Deadlock-related sync issues

- **Spinlock:** busy waiting; good for very short critical sections on multiprocessors.
- **Priority inversion:** a high-priority task waits for a lock held by a low-priority one, which is preempted by a medium-priority task. Fix: **priority inheritance**.

Unit 5: Deadlocks

5.1 Definition

A set of processes is deadlocked when each is waiting for a resource held by another in the set, so none can proceed.

5.2 Four necessary conditions (Coffman), all must hold

1. **Mutual exclusion:** resources cannot be shared.
2. **Hold and wait:** a process holds resources while waiting for others.
3. **No preemption:** resources cannot be forcibly taken.
4. **Circular wait:** a cycle of processes, each waiting for a resource held by the next.

5.3 Resource Allocation Graph (RAG)

Nodes: processes and resources. Edges: request (P to R) and assignment (R to P).

- No cycle: no deadlock.
- Cycle with **single-instance** resources: deadlock.
- Cycle with multi-instance resources: possible deadlock.

5.4 Handling deadlocks

1. **Prevention:** break at least one condition (e.g., request all resources at once; impose a **global resource ordering** to stop circular wait; allow preemption).
2. **Avoidance:** the OS uses advance information to stay in **safe states** (Banker's algorithm).
3. **Detection and recovery:** allow deadlock, detect it, then recover (kill processes, preempt resources, rollback).
4. **Ignorance (ostrich algorithm):** assume it is rare (used in Unix and Windows).

5.5 Banker's algorithm

Data structures, for n processes and m resource types:

- **Available[m]**
- **Max[n][m]**
- **Allocation[n][m]**
- **Need[n][m] = Max - Allocation**

Safety algorithm:

1. Work = Available; Finish[i] = false for all i.
2. Find i with Finish[i] = false and Need[i] <= Work.
3. Work += Allocation[i]; Finish[i] = true; repeat step 2.
4. If all Finish are true, the state is **safe** (a safe sequence exists).

A request is granted only if the resulting state remains safe.

Example: Available = (3, 3, 2)

P	Allocation	Max	Need
P0	0 1 0	7 5 3	7 4 3
P1	2 0 0	3 2 2	1 2 2
P2	3 0 2	9 0 2	6 0 0
P3	2 1 1	2 2 2	0 1 1
P4	0 0 2	4 3 3	4 3 1

- P1 (need 1 2 2 <= 3 3 2): run, Work = (5, 3, 2)
- P3 (0 1 1): Work = (7, 4, 3)
- P4 (4 3 1): Work = (7, 4, 5)
- P0 (7 4 3): Work = (7, 5, 5)
- P2 (6 0 0): Work = (10, 5, 7)

Safe sequence: **P1, P3, P4, P0, P2**.

5.6 Deadlock detection and recovery

Detection uses a wait-for graph (cycle means deadlock) or a Banker-like algorithm. Recovery: terminate all or one-by-one deadlocked processes, or preempt resources and roll back. Starvation can occur if the same victim is always chosen.

Unit 6: Memory Management

6.1 Basics

Memory is divided between the OS and user processes. Programs use **logical (virtual) addresses**; the **MMU** translates them to **physical addresses**.

Address binding can occur at compile time, load time, or **execution time** (needed for relocation).

Base and limit registers enforce protection: a process may access only addresses from `base` to `base + limit`.

6.2 Swapping

Moving processes between main memory and a backing store (disk) to free memory.

6.3 Contiguous allocation

- **Fixed (static) partitions:** causes **internal fragmentation** (wasted space inside a partition).
- **Variable (dynamic) partitions:** causes **external fragmentation** (free memory exists but is scattered).

Placement strategies: First fit (first hole that fits; fast), **Best fit** (smallest hole that fits; leaves tiny holes), **Worst fit** (largest hole).

Compaction shifts processes to merge free holes (expensive; needs execution-time binding).

6.4 Paging

Physical memory is divided into fixed-size **frames**; logical memory into **pages** of the same size. No external fragmentation; there can be small internal fragmentation in the last page.

Address = (page number p, offset d).

- Page number = logical address / page size
- Offset = logical address mod page size
- Physical address = frame number x page size + offset

Example: page size 1 KB, logical address 3500. p = 3, d = 3500 - 3072 = 428. If page 3 is in frame 7, physical address = 7 x 1024 + 428 = 7596.

If logical address is m bits and page size is 2^n, then page number has m - n bits and offset has n bits.

Page table: one per process; maps page to frame, with valid/invalid, protection, dirty and reference bits.

TLB (Translation Lookaside Buffer): a small fast cache of page-table entries.

Effective Access Time (EAT) = h x (t_TLB + t_mem) + (1 - h) x (t_TLB + 2 x t_mem), where h is hit ratio.

Example: TLB 20 ns, memory 100 ns, h = 0.8: EAT = 0.8 x 120 + 0.2 x 220 = 96 + 44 = **140 ns**.

Page table structures for large address spaces: hierarchical (multi-level), hashed, inverted page tables.

6.5 Segmentation

Memory is divided by **logical units** (code, stack, heap) of varying size. Address = (segment number, offset), checked against the segment's limit. Can cause external fragmentation. **Segmented paging** combines both.

Paging	Segmentation
Fixed-size pages	Variable-size segments
Invisible to programmer	Visible to programmer
Internal fragmentation	External fragmentation

Unit 7: Virtual Memory

7.1 Concept

Allows execution of processes **not completely in memory**, so programs can be larger than physical memory. Implemented with **demand paging**: a page is loaded only when it is needed.

7.2 Page fault handling

1. CPU references a page; the page table shows it is **invalid**, so a trap occurs.
2. OS finds the page on disk.
3. OS finds a free frame (or **replaces** a page).
4. Page is read into the frame.
5. Page table is updated; the instruction restarts.

Effective access time = (1 - p) x memory access + p x page-fault service time, where p is the page-fault rate. Because disk access is millions of times slower, p must be extremely small.

7.3 Page replacement algorithms

When no frame is free, choose a **victim** page.

FIFO: evict the oldest page. Suffers from **Belady's anomaly** (more frames can cause more faults).

Optimal (OPT): evict the page not used for the longest time in the future. Lowest faults; impossible to implement, used as a benchmark.

LRU (Least Recently Used): evict the page unused for the longest time in the past. Good approximation of OPT; no Belady's anomaly. Needs hardware support (counters or stack).

Clock (second chance): FIFO with a reference bit; gives pages a second chance.

LFU / MFU: based on use count.

Example: reference string `7 0 1 2 0 3 0 4 2 3 0 3 2`, 3 frames.

Algorithm	Page faults
FIFO	10
LRU	9
OPT	7

(For the classic string `7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1` with 3 frames: FIFO = 15, LRU = 12, OPT = 9.)

7.4 Frame allocation

Equal, proportional, or priority-based. **Global replacement** (victim from any process) vs **local replacement** (own frames only).

7.5 Thrashing

When a process lacks enough frames, it faults constantly and spends more time **paging than executing**, so CPU utilization collapses.

Fixes: the **working-set model** (keep each process's recently used pages in memory), **page-fault frequency** control, reducing the degree of multiprogramming.

7.6 Other topics

- **Copy-on-write:** parent and child share pages after `fork()`; a page is copied only when written.
- **Memory-mapped files.**
- **Locality of reference:** programs tend to reuse recent data and instructions (temporal) and nearby ones (spatial), which is why paging works.

Unit 8: File Systems

8.1 File concepts

A file is a named collection of related information on secondary storage.

Attributes: name, type, location, size, protection, timestamps, owner.

Operations: create, open, read, write, seek, delete, truncate.

Access methods: sequential, direct (random), indexed.

8.2 Directory structures

Single-level, two-level, **tree-structured** (most common), acyclic graph (links/shared files), general graph.

Links: a **hard link** is another directory entry for the same inode; a **symbolic (soft) link** is a file that stores a path.

8.3 File allocation methods

Method	How	Pros	Cons
Contiguous	Consecutive blocks	Fast sequential and direct access	External fragmentation, hard to grow
Linked	Each block points to the next	No external fragmentation	Slow random access, pointer overhead, reliability
Indexed	An index block holds all block pointers	Fast direct access	Index block overhead

FAT is a linked-allocation variant with the chain in a table. **Unix inode** uses direct pointers plus single, double and triple indirect blocks.

Max file size with inode: direct + (block size / pointer size) + (block size / pointer size)^2 + (block size / pointer size)^3 blocks.

8.4 Free-space management

Bitmap (bit vector), linked list, grouping, counting.

8.5 Protection and permissions

Unix permissions: read (r), write (w), execute (x) for **owner, group, others**. Example: `rwxr-xr--` = 754. **Access control lists** provide finer control.

8.6 File system layers and mounting

Application, logical file system, file-organization module, basic file system, I/O control (device drivers), devices. **Mounting** attaches a file system to the directory tree. **Journaling** (ext4, NTFS) logs changes so the file system can recover from crashes.

Unit 9: I/O and Disk Management

9.1 I/O methods

- **Programmed I/O (polling):** CPU repeatedly checks the device status; wasteful.
- **Interrupt-driven I/O:** device interrupts the CPU when ready.
- **DMA (Direct Memory Access):** a controller transfers blocks between device and memory without the CPU; the CPU is interrupted only when the transfer is done.

9.2 I/O software concepts

Buffering (smooth speed differences), **caching**, **spooling** (queue output for a device like a printer), device drivers.

9.3 Disk structure

Platters, tracks, sectors, cylinders.

Disk access time = seek time + rotational latency + transfer time.

- Average rotational latency = half of one rotation time.
- Example: 7200 RPM means one rotation = 8.33 ms, so average latency = 4.17 ms.

9.4 Disk scheduling

Goal: minimize seek time. Example: head at 53, requests `98, 183, 37, 122, 14, 124, 65, 67`.

Algorithm	Idea	Total head movement
FCFS	In arrival order	640
SSTF	Closest request next (may starve far requests)	236
SCAN (elevator)	Move one direction to the end, then reverse	depends on direction and disk size
C-SCAN	Scan one way, jump back to start, scan again	More uniform wait
LOOK / C-LOOK	Like SCAN / C-SCAN but go only as far as the last request	Better than SCAN

FCFS check: |98-53| + |183-98| + |37-183| + |122-37| + |14-122| + |124-14| + |65-124| + |67-65| = 45 + 85 + 146 + 85 + 108 + 110 + 59 + 2 = 640.

SSTF order: 53, 65, 67, 37, 14, 98, 122, 124, 183 with movement 12 + 2 + 30 + 23 + 84 + 24 + 2 + 59 = 236.

9.5 RAID

Level	Idea	Fault tolerance
RAID 0	Striping	None, fastest
RAID 1	Mirroring	Survives one disk failure
RAID 5	Striping with distributed parity	One disk
RAID 6	Double parity	Two disks
RAID 10	Mirror and stripe	Good performance and reliability

Unit 10: Protection and Security

10.1 Goals and principles

Protection controls access to resources; security defends against external and internal attacks. **Principle of least privilege:** give only the access needed.

10.2 Access matrix

Rows = domains (users), columns = objects, entries = rights. Implemented as **access control lists (per object)** or **capability lists (per domain)**.

10.3 Authentication

Passwords (stored as salted hashes), multi-factor authentication, biometrics, tokens.

10.4 Threats

Malware (virus, worm, trojan, ransomware), **buffer overflow**, privilege escalation, denial of service, spyware, insider threats.

10.5 Defenses

Firewalls, antivirus, patching, encryption, ASLR and non-executable stacks, sandboxing, auditing and logging.

Unit 11: Case Study: Linux Basics

11.1 Directory layout

`/` root, `/bin` commands, `/etc` configuration, `/home` user files, `/var` logs and variable data, `/tmp` temporary files, `/dev` devices, `/proc` process and kernel info.

11.2 Common commands

```
ls -l           # list with permissions
cd, pwd, mkdir, rm -r, cp, mv
cat, head, tail -f, grep, find
ps aux, top, kill -9 <pid>
chmod 755 file   # change permissions
chown user:group file
df -h, du -sh    # disk usage
```

11.3 Processes in Linux

Created with `fork()`/`clone()` and `exec()`. The scheduler (CFS, the Completely Fair Scheduler) uses a red-black tree to run the process with the smallest virtual runtime. Memory uses paging with demand paging and copy-on-write.

Quick Reference: Formulas

Topic	Formula
Turnaround time	CT - AT
Waiting time	TAT - BT
Response time	First CPU time - AT
Need matrix	Max - Allocation
Page number / offset	addr / page size, addr mod page size
Number of pages	Logical address space / page size
TLB EAT	h(t + m) + (1 - h)(t + 2m)
Demand-paging EAT	(1 - p) x m + p x fault time
Disk access time	seek + rotational latency + transfer
Page table size	Number of pages x entry size
Processes after n forks	2^n total

Common Interview and Exam Questions

1. Difference between **process and thread**.
2. What is a **context switch** and why is it costly?

3. Explain the four **Coffman conditions** and how to prevent deadlock.
4. **Mutex vs semaphore** vs monitor.
5. What is **virtual memory**, and how do page faults work?
6. What is **thrashing**, and how can it be avoided?
7. Compare **paging and segmentation**.
8. Explain **Belady's anomaly**.
9. What are **zombie and orphan processes**?
10. Compare the **CPU scheduling** algorithms.
11. What is **internal vs external fragmentation**?
12. What happens when you run a program (from `fork` to `exec` to execution)?
13. Explain **race condition** and how to prevent it.
14. What is **priority inversion**?
15. What are **system calls**, and how does user-to-kernel transition work?

Practice Problems with Answers

P1. Round Robin, quantum = 2. P1 (AT 0, BT 4), P2 (AT 1, BT 3), P3 (AT 2, BT 2).

Timeline: P1 (0-2), P2 (2-4), P3 (4-6, done), P1 (6-8, done), P2 (8-9, done).

Ready queue behavior: at t=2 queue is P2, P3, P1 (P1 arrives back after P3 arrival).

P	CT	TAT	WT
P1	8	8	4
P2	9	8	5
P3	6	4	2

Average WT = (4 + 5 + 2) / 3 = **3.67**.

P2. A system has 32-bit logical addresses, 4 KB pages and 4-byte page table entries. Page table size?

Pages = 2^32 / 2^12 = 2^20. Table size = 2^20 x 4 B = **4 MB**.

P3. How many total processes after this code runs?

```
fork(); fork(); fork();
```

Total = 2^3 = 8, so 7 new child processes.

P4. Three frames, string `1 2 3 4 1 2 5 1 2 3 4 5`. FIFO faults?

With 3 frames: **9 faults**. With 4 frames: **10 faults**. This demonstrates **Belady's anomaly**.

P5. Is this state safe? (Use Banker's algorithm from section 5.5.) Yes, with the sequence P1, P3, P4, P0, P2.

End of notes.